

Auditoría Exhaustiva de Modelos de Inteligencia Artificial para la Ciberseguridad

En esta sesión, abordaremos el tema crucial de cómo **auditar modelos de Inteligencia Artificial (IA)**, siguiendo un proceso estructurado desde la recopilación de datos hasta la mitigación de riesgos. Esta fase teórica sentará las bases para un posterior ejercicio práctico.

Antes de comenzar, es fundamental establecer que la **transparencia** y la **responsabilidad** en los modelos de IA son pilares para garantizar la confianza y la comprensión pública de cómo funcionan estos sistemas. La transparencia implica la apertura sobre el proceso de toma de decisiones del modelo, mientras que la responsabilidad implica que los creadores y usuarios son responsables de sus resultados, impulsando la adopción de prácticas éticas y la mitigación de riesgos.

Auditar modelos de IA en ciberseguridad es esencial para garantizar su **eficacia e integridad** en la detección de amenazas y ataques. En un mundo cada vez más conectado y digitalizado, los modelos de IA desempeñan un papel fundamental en la protección de sistemas y datos sensibles, por lo que su auditoría es vital para prevenir fugas de datos o problemas de privacidad.

1. Detección de Sesgos y Cumplimiento Regulatorio

La auditoría debe priorizar aspectos éticos y legales que impactan directamente en la equidad y la privacidad del usuario.

- **Detección de Sesgos y Problemas de Equidad:** Es esencial garantizar que los modelos de IA no perpetúen o amplifiquen las injusticias sociales. Los sesgos en los datos de entrenamiento pueden conducir a decisiones discriminatorias. La identificación y mitigación de estos sesgos aseguran que los sistemas traten a todas las personas de manera justa y equitativa,

independientemente de características como raza, género o la orientación sexual, por ejemplo.

- **Ciberseguridad y Protección de Datos:** Esto implica proteger la información sensible y prevenir ataques en los sistemas de IA. Es crucial asegurar el cumplimiento de las regulaciones de privacidad aplicables y la fiabilidad del modelo.
- **Fiabilidad y Consistencia:** La auditoría se enfoca en mantener la **precisión** y la **consistencia** del modelo a lo largo del tiempo, algo vital para un uso efectivo y confiable en la detección de amenazas.

2. Recorrido por las Fases de la Auditoría del Modelo

La auditoría de un modelo de IA se realiza a través de un *pipeline* estructurado en varias fases críticas.

2.1. Obtención de Datos de Entrenamiento

El primer paso es la **obtención y revisión de los datos** utilizados para el entrenamiento. Se debe recopilar el conjunto de datos utilizado por el modelo de IA y asegurarse de que los datos hayan sido recopilados de **forma ética** y que cumplan con las **regulaciones de privacidad adecuadas** y aplicables.

2.2. Identificación y Documentación de Modelos

Una vez revisado el *dataset*, el siguiente paso es la identificación de modelos utilizados en el entorno. Es necesario registrar y documentar los detalles sobre la **arquitectura del modelo** (el tipo de red neuronal, por ejemplo), los **hiperparámetros** utilizados y las **versiones** de las librerías empleadas.

2.3. Pruebas de Robustez y Seguridad (*Red Team Phase*)

Esta fase es similar a un ejercicio de *Red Team*, donde se ataca el modelo para evaluar su resistencia y seguridad:

- **Evaluación de Ataques Adversarios (*Adversarial Attack Evaluation*):** Esta prueba evalúa la **resistencia del modelo a manipulaciones maliciosas**. Esto puede incluir la inserción de "ruido" sutil en los datos de

entrada para forzar una clasificación incorrecta. Existen librerías especializadas que permiten automatizar estos ataques, como el **IBM Adversarial Robustness Toolbox (ART)**.

- **Verificación de la Privacidad:** Se debe asegurar que el modelo cumple con las regulaciones de privacidad, lo cual es especialmente importante si el modelo maneja datos sensibles.
- **Prueba de Estrés (*Stress Testing*):** Se fuerza al modelo para que maneje **cargas de trabajo inusuales** o en condiciones extremas. Esto incluye la evaluación de su capacidad de respuesta y su rendimiento bajo una alta exigencia, como un ataque de denegación de servicio (*Denial of Service*).

2.4. Validación de Resultados y Cross Validation

La fase final se enfoca en la validación de las salidas del modelo:

- **Comparación con Métricas de Referencia:** Se compara el rendimiento del modelo con métricas de referencia establecidas previamente, como, por ejemplo, las **tasas de detección de amenazas** en el contexto de la ciberseguridad. Esto permite evaluar si el modelo supera los estándares de seguridad establecidos.
- **Aplicación de Validación Cruzada (*Cross Validation*):** Se utiliza la técnica de validación cruzada para garantizar que el modelo **generaliza correctamente** a datos no vistos y que **no tiene un sobreajuste (*overfitting*)** con los datos de entrenamiento.

3. Herramientas y Marco de Colaboración

La auditoría requiere el uso de herramientas específicas y un enfoque de colaboración multidisciplinar.

3.1. Herramientas y Soluciones para la Auditoría

La selección de herramientas dependerá del problema, pero se pueden utilizar soluciones como:

- **Frameworks de Auditoría de IA:** El **IBM Adversarial Robustness Toolbox (ART)** es un ejemplo, ya que evalúa la robustez del modelo ante ataques adversarios.
- **Análisis de Interpretabilidad:** Se utilizan técnicas como **SHAP** (*SHapley Additive exPlanations*) para entender **cómo el modelo toma decisiones** y qué características de los datos son importantes para dicha decisión.
- **Plataformas de Pruebas de Seguridad y Auditoría de Código:** Estas herramientas son más habituales en un entorno de ciberseguridad, donde se verifica si el código subyacente del modelo tiene algún tipo de vulnerabilidad que lo haga susceptible a ataques externos, como inyección SQL (*SQL Injection*) o ataques de denegación de servicio.

3.2. Implicaciones y Colaboración Continua

Una vez que la auditoría ha finalizado, se establecen las conclusiones e implicaciones:

- **Mitigación de Riesgos:** La auditoría de modelos de IA, orientados a ciberseguridad, permite **identificar y mitigar riesgos potenciales**, lo cual es crucial para proteger los sistemas y datos sensibles en general.
- **Mejora Continua (*Continuous Improvement*):** La auditoría debe ser un **proceso continuo y cíclico** para garantizar que los modelos de ciberseguridad se mantengan actualizados y efectivos en un entorno que está en constante cambio.
- **Colaboración Interdisciplinaria:** Es fundamental trabajar en conjunto con diferentes **equipos, organismos o departamentos** de la organización para garantizar la protección de datos y sistemas, lo cual involucra a todo el mundo.

4. Conclusión Final

La auditoría de modelos de IA es **fundamental** para garantizar su **fiabilidad, transparencia y cumplimiento de estándares éticos**. Este proceso ayuda a detectar y mitigar sesgos, asegura la seguridad de los datos y mantiene la responsabilidad en los procesos de toma de decisiones de la IA.

Al evaluar regularmente los modelos, las organizaciones pueden identificar posibles problemas de equidad, encontrar fallos de vulnerabilidades de seguridad y asegurar el cumplimiento de regulaciones de protección de datos. Todo esto contribuye a crear sistemas de IA responsables que generen mayor confianza y respeten los derechos individuales.

Auditoría Práctica de Modelos de IA

La sesión de hoy se centrará en la **auditoría práctica de modelos de IA**. Específicamente, simularemos la auditoría de un modelo diseñado para **detectar correos electrónicos de phishing**.

Este enfoque nos permitirá aplicar los conocimientos teóricos previos para evaluar la efectividad y solidez de un modelo de *Machine Learning*.

El contexto de este ejercicio es la **simulación de una auditoría** para un modelo de IA en la detección de correos de *phishing*. El objetivo es verificar la efectividad del modelo y entender cómo se introducen los conceptos de auditoría en la práctica.

Este primer bloque de código se centra en la **carga** del *dataset* de correos electrónicos y la **inspección inicial** de su estructura y contenido.

```
# load the dataset
import pandas as pd
data_path = 'emails_auditing.csv'
emails_df = pd.read_csv(data_path, encoding='latin1')

# Display basic information and the first few rows of the
dataset
print(emails_df.info())
print(emails_df.head())
```

Sintaxis y Propósito Línea por Línea

1. **# load the dataset**: Este es un comentario en Python que sirve como título o encabezado, indicando que el bloque de código subsiguiente se encarga de cargar el conjunto de datos.
2. **import pandas as pd**: Esta línea importa la librería **Pandas**, que es la herramienta fundamental para el manejo y análisis de datos estructurados. Se le asigna el alias **pd** para facilitar su uso posterior en el código.

3. `data_path = 'emails_auditing.csv'`: Se define una variable llamada `data_path` que almacena la ruta o el nombre del archivo CSV (`emails_auditing.csv`) que contiene el *dataset*.
4. `emails_df = pd.read_csv(data_path, encoding='latin1')`: Esta es la línea de **carga de datos**. Utiliza la función `read_csv` de Pandas para leer el archivo especificado en `data_path`. La lectura se realiza con la codificación `'latin1'` para asegurar que los caracteres especiales se interpreten correctamente. El resultado se guarda en un DataFrame llamado `emails_df`.
5. `# Display basic information and the first few rows of the dataset`: Este comentario indica que el código a continuación tiene fines de inspección y visualización inicial.
6. `print(emails_df.info())`: Llama al método `.info()` del DataFrame. Su propósito es mostrar un resumen conciso de los datos, incluyendo el total de entradas (957), el número de columnas (3), los tipos de datos de cada columna (ej., `int64` para números, `object` para texto), y si hay valores nulos (*non-null*).
7. `print(emails_df.head())`: Llama al método `.head()`. Su propósito es imprimir las primeras cinco filas del DataFrame, permitiendo dar un vistazo rápido a los contenidos reales del *dataset* (el cuerpo del mensaje y la etiqueta).

Análisis del Output de la Terminal

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 957 entries, 0 to 956
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   S. No.          957 non-null   int64
1   Message_body    957 non-null   object
2   Label           957 non-null   object
dtypes: int64(1), object(2)
memory usage: 22.6+ KB
None
```

	S. No.	Message_body	Label
0	1	Rofl. Its true to its name	Non-Spam
1	2	The guy did some bitching but I acted like i'd...	Non-Spam
2	3	Pity, * was in mood for that. So...any other s...	Non-Spam
3	4	Will ü b going to esplanade fr home?	Non-Spam
4	5	This is the 2nd time we have tried 2 contact u...	Spam

El *output* de la terminal se divide en dos partes principales, correspondientes a las llamadas a `.info()` y `.head()`:

1. Salida de `emails_df.info()` (Resumen de Datos)

Esta sección ofrece un resumen conciso de los datos:

- **Total de Entradas:** Muestra **957 entradas**, enumeradas del 0 al 956, lo que indica el número total de correos electrónicos en el conjunto de datos.
- **Columnas:** Hay un total de **3 columnas**.
 - **S. No. (Número de Serie):** Es de tipo numérico (`int64`).
 - **Message_body (Cuerpo del Mensaje):** Es de tipo texto (`object`).
 - **Label (Etiqueta):** Es de tipo texto (`object`) y contiene el valor final que clasifica el correo como `Spam` o `Non-Spam`.
- **Valores Nulos:** Todas las columnas (`S. No.`, `Message_body`, `Label`) tienen **957 valores non-null**, lo que implica que no hay **datos faltantes** en el *dataset*.
- **Uso de Memoria:** El DataFrame ocupa aproximadamente **22.6 KB** de memoria.

2. Salida de `emails_df.head()` (Vista Previa de Filas)

Esta sección imprime las **primeras cinco filas** del DataFrame. Esto es útil para obtener una comprensión inicial de cómo se estructuran los datos, mostrando ejemplos de correos electrónicos etiquetados como *spam* o *non-spam*. Se puede observar que los cuerpos de los mensajes son cadenas de texto y que la etiqueta es **Non-Spam** en las filas mostradas.

```
# Basic statistics
print(emails_df.describe())

# Check for missing values
print(emails_df.isnull().sum())

# Visualization of label distribution
import matplotlib.pyplot as plt

emails_df['Label'].value_counts().plot(kind='bar')
plt.title('Distribution of Email Labels')
plt.xlabel('Label')
plt.ylabel('Count')
plt.show()
```

Este bloque de código se centra en la **auditoría inicial de la calidad de los datos** al verificar la ausencia de valores nulos y en la **visualización de la distribución** de las etiquetas de *spam*.

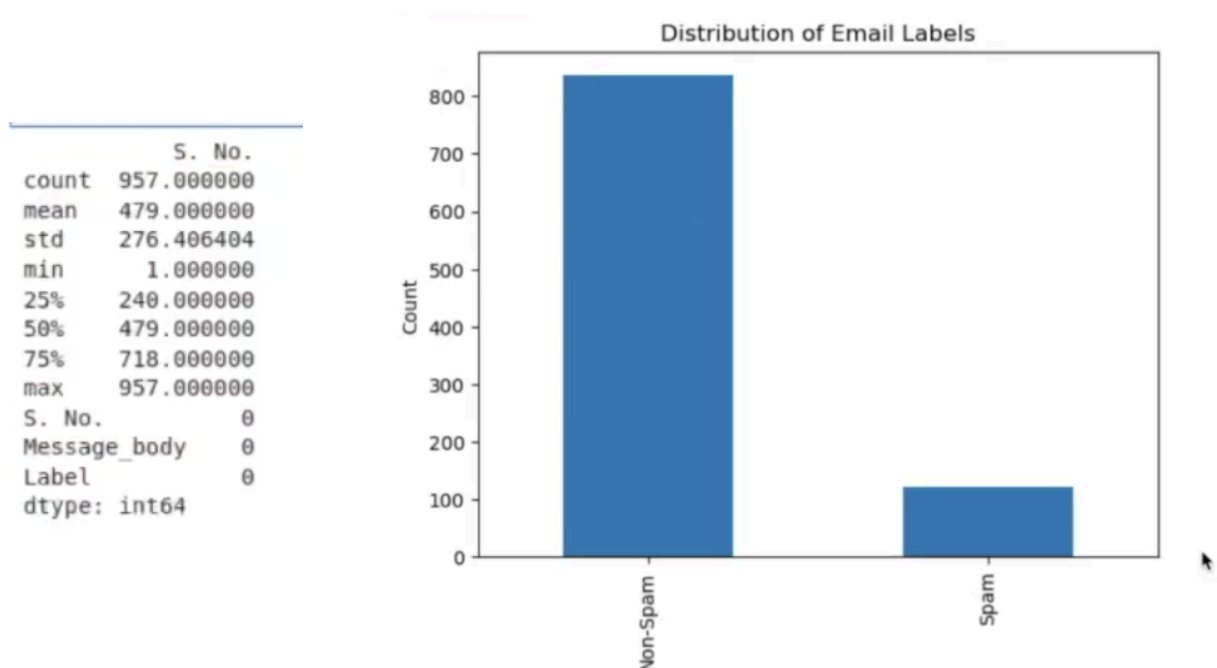
Sintaxis y Propósito Línea por Línea

1. **# Basic statistics**: Comentario que indica la intención de generar estadísticas básicas.
2. **print(emails_df.describe())**: Utiliza el método **.describe()** de Pandas para generar estadísticas descriptivas (como la tendencia central, dispersión, etc.). En este caso, solo afecta a la columna numérica **S. No.**, ya que las otras son de tipo texto.

3. **# Check for missing values**: Comentario que señala la comprobación de valores faltantes.
4. **print(emails_df.isnull().sum())**: Esta línea es crucial para la integridad de los datos. El método **.isnull()** crea una matriz booleana indicando dónde hay valores nulos, y **.sum()** calcula la suma de los valores nulos para cada columna. Esto proporciona una verificación rápida de la integridad de los datos.
5. **# Visualization of label distribution**: Comentario que marca la intención de visualizar la distribución de las etiquetas.
6. **import matplotlib.pyplot as plt**: Importa la librería **Matplotlib** con el alias **plt**, que se utilizará para crear la gráfica de barras.
7. **emails_df['Label'].value_counts().plot(kind='bar')**: Esta es la línea de generación del gráfico. **.value_counts()** cuenta el número de correos electrónicos en cada categoría de etiqueta (**Spam** o **Non-Spam**), y **.plot(kind='bar')** genera un gráfico de barras con esos conteos.
8. **plt.title(...), plt.xlabel(...), plt.ylabel(...)**: Estos comandos definen los títulos y etiquetas de los ejes para hacer el gráfico interpretable.
9. **plt.show()**: Muestra la gráfica de barras en la salida.

Análisis del Output de la Terminal

El *output* consta de tres partes: las estadísticas descriptivas, la verificación de valores nulos y la gráfica de distribución.



1. Estadísticas Descriptivas (`describe()`)

La salida del `.describe()` se centra en la columna `S. No.`. Muestra el recuento total (`count`: 957), la media (`mean`: 479.0), la desviación estándar (`std`), y los valores de cuartiles (mínimo, 25%, 50%, 75%, máximo). Estos datos confirman la cantidad de entradas en el *dataset*.

2. Verificación de Valores Nulos (`isnull().sum()`)

La salida de `isnull().sum()` para todas las columnas es **cero (0)**. Esto **confirma la integridad de los datos** y asegura que no hay valores faltantes en las columnas `S. No.`, `Message_body`, o `Label`. Esta verificación es una buena práctica de auditoría.

3. Gráfico de Distribución de Etiquetas

La gráfica de barras generada, titulada **"Distribution of Email Labels"**, muestra una clara **asimetría o desequilibrio de clases**:

- **Clase Mayoritaria (Non-Spam)**: El número de correos clasificados como `Non-Spam` es significativamente alto, superando las **800** ocurrencias.
- **Clase Minoritaria (Spam)**: El número de correos clasificados como `Spam` es bajo, aproximadamente **120** ocurrencias.

Implicación para la Auditoría: Este **desequilibrio de clases** es un hallazgo crítico para la auditoría. Un modelo entrenado con este *dataset* tenderá a estar sesgado hacia la clase mayoritaria (`Non-Spam`). Podría lograr una alta **precisión (Accuracy)** simplemente prediciendo "Non-Spam" todo el tiempo, lo que enmascararía un rendimiento deficiente en la detección real de *phishing* (la clase minoritaria `Spam`). Esto sugiere que, para una auditoría efectiva, será necesario evaluar el modelo utilizando métricas sensibles al desequilibrio, como el **Recall** y el **F1 Score**.

```

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import LabelEncoder

# Encoding labels
le = LabelEncoder()
emails_df['label_encoded'] =
le.fit_transform(emails_df['Label'])

# Splitting dataset
X_train, X_test, y_train, y_test = train_test_split(
    emails_df['Message_body'], emails_df['label_encoded'],
    test_size=0.2, random_state=42
)

# Text vectorization
tfidf_vectorizer = TfidfVectorizer(stop_words='english',
max_features=1000)
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

```

Este bloque de código es fundamental, ya que prepara el *dataset* de correos electrónicos para que los algoritmos de *Machine Learning* lo puedan procesar. Esto se logra en tres pasos críticos: la **codificación de etiquetas**, la **división de datos** y la **vectorización del texto**.

Sintaxis y Propósito Línea por Línea

1. **from sklearn.model_selection import train_test_split:**
 Importa la función `train_test_split` de Scikit-learn, que es esencial para dividir el *dataset* en conjuntos de entrenamiento y prueba.
2. **from sklearn.feature_extraction.text import TfidfVectorizer:** Importa el `TfidfVectorizer` de Scikit-learn. Esta clase se utiliza para convertir el texto crudo en una matriz de características numéricas mediante la técnica TFIDF.

3. **from sklearn.preprocessing import LabelEncoder**: Importa la clase **LabelEncoder** de Scikit-learn. Su propósito es convertir las etiquetas categóricas (como 'spam' o 'no spam') en valores numéricos, ya que la mayoría de los modelos de *Machine Learning* solo pueden procesar datos numéricos.
-

Codificación de Etiquetas (**LabelEncoder**)

4. **# Encoding labels**: Comentario que indica el inicio del proceso de codificación de etiquetas.
 5. **le = LabelEncoder()**: Crea una instancia del codificador de etiquetas.
 6. **emails_df['label_encoded'] = le.fit_transform(emails_df['Label'])**: Aplica el proceso de codificación. El método **.fit_transform()** se ajusta a los datos de la columna **'Label'** y transforma las etiquetas de texto (ej. 'spam', 'no spam') en un valor entero único (ej. 0 y 1). Esta nueva representación numérica se guarda en una columna llamada **'label_encoded'**.
-

División del Dataset (**train_test_split**)

7. **# Splitting dataset**: Comentario que indica el inicio de la división de datos.
8. **X_train, X_test, y_train, y_test = train_test_split(...)**: Esta línea divide el *dataset* en cuatro subconjuntos:
 - **emails_df['Message_body']**: Se utiliza como la característica (X) a predecir.
 - **emails_df['label_encoded']**: Se utiliza como la etiqueta objetivo (y).
 - **test_size=0.2**: Indica que el **20% de los datos** se reservará para el conjunto de prueba.

- **random_state=42**: Este parámetro garantiza que la división sea **reproducible**; es decir, que siempre se obtenga el mismo resultado al ejecutar el código.
-

Vectorización del Texto (TfidfVectorizer)

9. **# Text vectorization**: Comentario que indica el inicio del proceso de vectorización.
10. **tfidf_vectorizer = TfidfVectorizer(stop_words='english', max_features=1000)**: Crea una instancia del vectorizador TFIDF.
 - **stop_words='english'**: Excluye palabras comunes en inglés (como 'the', 'is') que tienen poco valor predictivo.
 - **max_features=1000**: Limita el vocabulario a los 1.000 términos más significativos, lo que reduce la dimensionalidad del modelo.
 - El concepto de TFIDF (Term Frequency Inverse Document Frequency) es una técnica estadística que pondera las palabras no solo por su frecuencia en un documento, sino también por su rareza en todo el conjunto de datos, ayudando a resaltar palabras informativas.
11. **X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)**:
Ajusta y Transforma los datos de entrenamiento (**X_train**). El método **.fit_transform()** calcula el vocabulario y los pesos TFIDF basándose *únicamente* en los datos de entrenamiento. Esta es una práctica clave para **evitar la filtración de información** del conjunto de prueba.
12. **X_test_tfidf = tfidf_vectorizer.transform(X_test)**:
Transforma los datos de prueba (**X_test**). Aquí solo se usa **.transform()** para aplicar la transformación aprendida en el paso anterior a los datos de prueba, asegurando que las representaciones numéricas sean consistentes para ambos conjuntos.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Training a simple logistic regression model
model = LogisticRegression()
model.fit(X_train_tfidf, y_train)

# Prediction and evaluation
y_pred = model.predict(X_test_tfidf)
print(f'Accuracy: {accuracy_score(y_test, y_pred)}')

# Output de la terminal:
# Accuracy: 0.9010416666666666
```

Análisis Sintáctico del Código

1. **from sklearn.linear_model import LogisticRegression**: Importa la clase **LogisticRegression**. Esta es la técnica de *Machine Learning* supervisado elegida para clasificar los *emails* en las categorías predefinidas de *spam* o *no spam*.
2. **from sklearn.metrics import accuracy_score**: Importa la función **accuracy_score**, que se utilizará para medir la precisión general del modelo.
3. **# Training a simple logistic regression model**: Comentario que indica el inicio del entrenamiento del modelo.
4. **model = LogisticRegression()**: Crea una instancia del modelo de Regresión Logística. Esto inicializa el modelo con sus parámetros por defecto y lo prepara para el entrenamiento.
5. **model.fit(X_train_tfidf, y_train)**: Esta es la línea de **entrenamiento**. El modelo aprende a asociar las características de los *emails* (**X_train_tfidf**, que son los datos transformados con TFIDF) con sus etiquetas reales (**y_train**). Durante este proceso, el modelo ajusta sus parámetros internos, como los pesos de las características, para minimizar la función de pérdida logística.

6. **# Prediction and evaluation:** Comentario que indica el inicio de la fase de prueba.
 7. **y_pred = model.predict(X_test_tfidf):** Se utiliza el método **.predict()** para aplicar los parámetros aprendidos a los datos de prueba (**X_test_tfidf**). El resultado es **y_pred**, el vector de etiquetas predichas por el modelo para el conjunto de prueba.
 8. **print(f'Accuracy: {accuracy_score(y_test, y_pred)}')**: Calcula y muestra la precisión. La función **accuracy_score** compara las etiquetas predichas (**y_pred**) con las etiquetas reales (**y_test**) del conjunto de prueba. La precisión mide el **porcentaje de emails clasificados correctamente** sobre el total de *emails* de prueba.
-

Análisis del Output de la Terminal

El *output* de la terminal proporciona la métrica de precisión calculada: **Accuracy: 0.9010416666666666**.

- **Interpretación:** La precisión es muy alta, llegando al **90%**. Esto implica que el modelo de Regresión Logística ha clasificado correctamente aproximadamente el **90% de los correos electrónicos** en el conjunto de prueba.
- **Contexto de Auditoría:** Si bien un 90% de precisión parece excelente, el análisis previo ya reveló un **fuerte desequilibrio de clases** en el *dataset* (muchos más correos 'No Spam' que 'Spam'). Por lo tanto, esta métrica de precisión es **engañosa**. El modelo podría estar logrando ese 90% simplemente por clasificar casi todo como 'No Spam'. Para una auditoría efectiva, es fundamental usar métricas que midan el rendimiento en la clase minoritaria (*spam* o *phishing*).


```
# Using a simple technique to interpret the model (feature
importance)
feature_importances = pd.DataFrame(model.coef_[0],

index=tfidf_vectorizer.get_feature_names_out(),

columns=['importance']).sort_values('importance',
ascending=False)
print(feature_importances.head(10))
```

Output de la terminal:

```
#          importance
# free          1.889500
# stop          1.884157
# claim         1.785859
# www           1.745451
# mobile        1.586987
# txt           1.561298
# prize         1.512023
# uk            1.365179
# cash          1.289190
# text          1.284641
```

Análisis Sintáctico del Código

1. **# Using a simple technique to interpret the model (feature importance)**: Este comentario indica el objetivo del bloque: auditar el modelo para comprender cuáles son los **términos o palabras más influyentes** en la clasificación.
2. **feature_importances = pd.DataFrame(...)**: Esta línea completa crea un nuevo DataFrame llamado **feature_importances** para organizar la información de una manera legible.
3. **model.coef_[0]**: Accede a los **coeficientes** del modelo de Regresión Logística. Estos coeficientes representan la **fuerza y la dirección** de la

relación entre cada característica (palabra) y la probabilidad de que un correo electrónico pertenezca a la clase positiva (en este caso, *spam*).

4. `index=tfidf_vectorizer.get_feature_names_out()`: Asigna las **palabras reales** (los *features*) que fueron extraídas por el `tfidf_vectorizer` como índices del nuevo DataFrame, lo que permite asociar cada coeficiente numérico con la palabra correspondiente.
5. `columns=['importance']`: Nombra la columna de coeficientes como `'importance'`.
6. `.sort_values('importance', ascending=False)`: Ordena el DataFrame resultante por la columna `'importance'` de forma descendente, colocando las palabras más influyentes en la parte superior.
7. `print(feature_importances.head(10))`: Imprime las **10 palabras más importantes** y sus valores de coeficiente.

Análisis del Output de la Terminal

El *output* de la terminal muestra las diez palabras con mayor valor de importancia para la clasificación, lo cual es fundamental para **auditar la plausibilidad del modelo**.

- **Términos Influyentes:** Palabras como **free**, **stop**, **claim**, **www**, **mobile**, **prize**, y **cash** aparecen en la parte superior de la lista.
- **Significado de los Valores:** Todos los valores de importancia son **positivos**. Esto indica una **asociación positiva con la clase objetivo (spam)**. La presencia de cualquiera de estas palabras en un correo electrónico incrementa la probabilidad de que el modelo lo clasifique como *spam*.
- **Validación de la Auditoría:** Esta interpretación es fundamental. Las palabras que el modelo considera más importantes son altamente **consistentes** con el lenguaje típico de los anuncios, esquemas de fraude y *phishing* (ofertas de "free cash", instrucciones de "stop" o "claim" y enlaces **www**). Esta consistencia sirve como una **validación clave** de que el modelo no solo toma decisiones, sino que también ha aprendido **patrones razonables y plausibles**.

- **Aplicación:** Esta información es útil para ajustar el modelo o crear estrategias de intervención si fuera necesario.

```
from sklearn.model_selection import cross_val_score

# Cross-validation to evaluate model generalization
scores = cross_val_score(model, X_train_tfidf, y_train, cv=5)
print(f'Cross-Validation Accuracy Scores: {scores}')
```

Output de la terminal:

```
Cross-Validation Accuracy Scores: [0.88235294 0.88235294
0.88888889 0.88888889 0.88888889]
```

Análisis Sintáctico del Código

1. **from sklearn.model_selection import cross_val_score:** Importa la función **cross_val_score**. Esta función de Scikit-learn es la encargada de ejecutar el procedimiento de **validación cruzada**.
2. **# Cross-validation to evaluate model generalization:** Comentario que indica el propósito principal: evaluar la **capacidad de generalización** del modelo.
3. **scores = cross_val_score(model, X_train_tfidf, y_train, cv=5):** Esta línea ejecuta la validación cruzada.
 - **model:** Es la instancia del modelo de Regresión Logística previamente entrenado.
 - **X_train_tfidf y y_train:** Son las características y etiquetas del conjunto de **entrenamiento**. La validación cruzada opera dividiendo estos datos.

- **cv=5**: Especifica el número de divisiones (*folds*). Esto indica que el conjunto de datos se dividirá en 5 subconjuntos, y la validación se repetirá 5 veces, rotando el subconjunto usado como prueba.
4. **print(f'Cross-Validation Accuracy Scores: {scores}')**:
- Imprime el vector de puntuaciones de precisión obtenido de las 5 iteraciones.

Análisis del Output de la Terminal

El *output* muestra las puntuaciones de precisión obtenidas para cada una de las cinco iteraciones de la validación cruzada.

- **Puntuaciones de Precisión**: Las puntuaciones son muy consistentes: [0.88235...,0.88235...,0.88888...,0.88888...,0.88888...].
- **Interpretación en Porcentaje**: La precisión lograda en cada una de las cinco iteraciones varía ligeramente, manteniéndose en un rango estrecho entre el **88.2% y el 88.9%**.
- **Conclusión de Robustez**: La **consistencia** en las puntuaciones a través de las diferentes iteraciones es una señal altamente positiva. Esto sugiere que el modelo es **estable** y posee una buena **capacidad de generalización** a nuevos datos. El modelo demuestra ser relativamente **insensible** a variaciones específicas en las muestras de datos de entrenamiento y prueba, lo cual es crucial para aplicaciones del mundo real.
- **Propósito de la Auditoría**: Estos resultados de *Cross-Validation* proporcionan una fuerte **confianza en la robustez** del modelo de Regresión Logística, indicando que es una herramienta fiable para la clasificación de correos electrónicos.

```
from sklearn.metrics import confusion_matrix,
classification_report

y_pred = model.predict(X_test_tfidf)
conf_matrix = confusion_matrix(y_test, y_pred)
print(conf_matrix)
print(classification_report(y_test, y_pred))
```

Análisis Sintáctico del Código

1. `from sklearn.metrics import confusion_matrix, classification_report`: Importa dos herramientas clave para la auditoría de clasificación:
 - `confusion_matrix`: Función que calcula la matriz de confusión.
 - `classification_report`: Función que genera un informe completo con las métricas de Precisión, Recall, F1 Score y Soporte, desglosadas por clase.
2. `y_pred = model.predict(X_test_tfidf)`: Esta línea **repite la predicción**, asegurando que el vector de etiquetas predichas (`y_pred`) esté disponible para las funciones de métricas.
3. `conf_matrix = confusion_matrix(y_test, y_pred)`: Calcula la **matriz de confusión** comparando las etiquetas reales del conjunto de prueba (`y_test`) con las etiquetas predichas (`y_pred`).
4. `print(conf_matrix)`: Imprime la matriz de confusión en formato matricial.
5. `print(classification_report(y_test, y_pred))`: Imprime el informe detallado de clasificación, que es una herramienta clave para la auditoría de modelos desequilibrados.

Análisis del Output de la Terminal

El *output* contiene dos secciones principales: la **Matriz de Confusión** y el **Reporte de Clasificación**.

```
[[165  0]
 [ 19  8]]
```

	precision	recall	f1-score	support
0	0.90	1.00	0.95	165
1	1.00	0.30	0.46	27
accuracy			0.90	192
macro avg	0.95	0.65	0.70	192
weighted avg	0.91	0.90	0.88	192

1. Matriz de Confusión

La matriz de confusión es una herramienta clave para la evaluación de modelos de clasificación. Muestra cómo se organizan las predicciones del modelo frente a las clases reales:

- **Verdaderos Negativos (165):** Indica que 165 correos electrónicos fueron **correctamente** clasificados como **No Spam** (clase 0).
- **Falsos Positivos (0):** Indica que **0** correos electrónicos No Spam fueron **incorrectamente** clasificados como Spam.
- **Falsos Negativos (19):** Indica que 19 correos electrónicos Spam fueron **incorrectamente** clasificados como **No Spam**.
- **Verdaderos Positivos (8):** Indica que 8 correos electrónicos fueron **correctamente** clasificados como **Spam** (clase 1).

El número de **Falsos Negativos (19)** es una alerta de seguridad, ya que son correos de *phishing* que el modelo **no detectó**.

2. Reporte de Clasificación

El reporte desglosa el rendimiento por clase (0: No Spam, 1: Spam). El soporte (**support**) indica el número de ocurrencias reales de la clase en el conjunto de prueba (165 para No Spam, 27 para Spam).

- **Clase 0 (No Spam):**
 - **Precisión (0.90):** El 90% de los correos que el modelo clasificó como No Spam eran realmente No Spam.
 - **Recall (1.00):** El 100% de los correos No Spam reales fueron identificados correctamente.
 - **Conclusión:** El modelo es **excelente** para identificar la clase mayoritaria (No Spam).
- **Clase 1 (Spam / Phishing):**
 - **Precisión (1.00):** El 100% de los correos que el modelo clasificó como Spam eran realmente Spam. Esto se debe a que no hubo Falsos Positivos (0).

- **Recall (0.30):** Solo el **30%** de los correos Spam reales fueron detectados. Esto confirma los **19 Falsos Negativos** de la matriz de confusión.
- **F1 Score (0.46):** Este valor bajo refleja la falta de equilibrio, ya que es el promedio armónico entre la Precisión y el Recall.

Conclusiones Finales de la Auditoría:

El análisis del reporte de clasificación y la matriz de confusión confirman que el modelo está **sesgado** hacia la clase mayoritaria. El modelo es **muy bueno para identificar correos de No Spam** (Precisión en clase 0 y *Weighted Average* altos), pero su **capacidad de detectar phishing real (Recall para clase 1)** es muy pobre (30%). Esto significa que es un sistema de seguridad **no fiable** para la detección de *phishing* en producción, ya que dejaría pasar al menos 7 de cada 10 ataques.

```
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# Asumiendo y_test y y_pred_prob como las etiquetas reales y
# las probabilidades predichas para la clase positiva
y_pred_prob = model.predict_proba(X_test_tfidf)[:, 1]
roc_auc = roc_auc_score(y_test, y_pred_prob)
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)

plt.figure()
plt.plot(fpr, tpr, label='Logistic Regression (area = %0.2f)'
% roc_auc)
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.show()
```

Análisis Sintáctico del Código

1. `from sklearn.metrics import roc_auc_score, roc_curve:`
Importa las funciones necesarias de Scikit-learn para calcular las métricas ROC:
 - `roc_auc_score`: Calcula el valor del Área Bajo la Curva (AUC).
 - `roc_curve`: Calcula los valores necesarios para dibujar la curva ROC: la Tasa de Falsos Positivos (*FPR*) y la Tasa de Verdaderos Positivos (*TPR*).
2. `import matplotlib.pyplot as plt`: Importa la librería Matplotlib, esencial para generar el gráfico de la curva ROC.
3. `y_pred_prob = model.predict_proba(X_test_tfidf)[: , 1]`: Esta línea es clave para la curva ROC. A diferencia de `model.predict()`, utiliza `.predict_proba()` para obtener las probabilidades de que cada instancia pertenezca a la clase positiva (Spam). El `[ : , 1]` selecciona solo la columna de probabilidad para la clase positiva.
4. `roc_auc = roc_auc_score(y_test, y_pred_prob)`: Calcula el valor **AUC**, que cuantifica la capacidad de distinción del modelo.
5. `fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)`:
Calcula los tres componentes necesarios para la curva: la Tasa de Falsos Positivos (`fpr`, eje X), la Tasa de Verdaderos Positivos (`tpr`, eje Y), y los umbrales de corte.
6. `plt.plot(fpr, tpr, ...)`: Dibuja la curva ROC con los valores calculados de `fpr` y `tpr`. La etiqueta incluye el valor AUC formateado.
7. `plt.plot([0, 1], [0, 1], 'r--')`: Dibuja una línea de referencia diagonal (`r--`) que representa un **modelo aleatorio** (un AUC de 0.5).
8. `plt.title('Receiver operating characteristic')`: Establece el título del gráfico. La curva ROC representa la relación entre la Tasa de Verdaderos Positivos (Recall) en el eje Y y la Tasa de Falsos Positivos (*FPR*) en el eje X.

Análisis del Output de la Terminal (Gráfico)

El gráfico resultante es la curva **Receiver Operating Characteristic (ROC)**.

- **Curva Azul (Rendimiento del Modelo):** La línea azul se eleva **rápidamente** hacia la esquina superior izquierda del gráfico. Esto indica que el modelo logra una **alta Tasa de Verdaderos Positivos** (alta sensibilidad o Recall) con una **baja Tasa de Falsos Positivos**.
- **Valor AUC (Área Bajo la Curva):** El valor de **AUC es 0.99**, lo cual está muy cerca del máximo valor posible de 1.0.
- **Interpretación de la Auditoría:** Un AUC de 0.99 significa que el modelo tiene un **rendimiento excelente** para distinguir entre correos legítimos y *phishing*. Esta métrica es una medida integral de la calidad de un clasificador binario.

Consideración de Seguridad: Aunque el AUC de 0.99 es excepcional, es fundamental recordar que **no debe ser el único criterio**. Debe complementarse con la auditoría de otras métricas sensibles al contexto (como Precisión, Recall) y con la validación de que el modelo **no esté sobreajustado (*overfitting*)** o sesgado por el desequilibrio de clases. Si el modelo pasa estas consideraciones adicionales, se puede considerar confiable.

Conclusiones Finales: Auditoría de Modelos de Clasificación

A través de este ejercicio práctico, hemos completado una auditoría exhaustiva de un modelo de **Regresión Logística** diseñado para la clasificación de *emails*. El uso de un conjunto de datos, aunque sencillo, nos ha permitido aplicar y verificar prácticas fundamentales de *Machine Learning* y auditoría.

Puntos Clave de la Auditoría

1. **Preprocesamiento y Vectorización:** Se implementaron prácticas estándar como la codificación de etiquetas (**LabelEncoder**) y el preprocesamiento de

texto mediante la técnica **TFIDF** (*Term Frequency-Inverse Document Frequency*).

2. **Evaluación Multi-Métrica:** El rendimiento del modelo se evaluó a través de múltiples métricas clave:
 - **Matriz de Confusión e Informe de Clasificación:** Estas herramientas proporcionaron una visión detallada del rendimiento, reafirmando la **alta efectividad del modelo para clasificar la clase mayoritaria** (*No Spam*). Sin embargo, revelaron un **área crítica de mejora** en la identificación de la clase minoritaria (*Spam*), evidenciando la necesidad de optimizar el **Recall** para la detección de *phishing* real.
 - **Curva ROC y Valor AUC:** La curva ROC, con un valor de **AUC cercano a la perfección**, reafirmó la robusta habilidad del modelo para distinguir entre las clases positiva y negativa.
3. **Interpretabilidad del Modelo:** Se demostró la importancia de la interpretabilidad al analizar la **influencia de las características**. Al identificar los términos más importantes (como 'free', 'claim', y 'www'), se validó que el modelo estaba aprendiendo patrones intuitivos y plausibles asociados al *phishing*.

En definitiva, este ejercicio subraya la necesidad crítica de un **enfoque equilibrado y multiangular en la auditoría de modelos**. Es fundamental balancear las métricas de rendimiento globales (como la Precisión) con la interpretabilidad del modelo y la equidad en la detección de clases minoritarias, garantizando que el sistema sea confiable y robusto para su implementación en un entorno de seguridad real.

Seguridad en el Machine Learning: El Proyecto

OMLASP y los Ataques Adversarios

En esta sesión, abordaremos la iniciativa **OMLASP** (*Open Machine Learning Application Security Project*), una colección de herramientas y una guía fundamental para auditar modelos y arquitecturas de *Machine Learning* (ML) e Inteligencia Artificial (IA).

El proyecto OMLASP surge de la necesidad de establecer **metodologías estandarizadas** para auditar algoritmos de ML, enfocándose en la identificación y mitigación de **vulnerabilidades de seguridad** y **sesgos**. La iniciativa no solo busca destacar las potenciales brechas de seguridad, sino que también proporciona un marco de trabajo para una **auditoría sistemática**. Su objetivo principal es concienciar a desarrolladores, especialistas en seguridad y expertos en IA sobre la importancia de incorporar **prácticas de seguridad** desde las fases tempranas del desarrollo de aplicaciones basadas en *Machine Learning*.

1. Seguridad e Integridad del Machine Learning

La seguridad en el *Machine Learning* es vital debido al papel fundamental que estas tecnologías desempeñan en la vida cotidiana. Aunque los algoritmos de *Machine Learning* son poderosos, son susceptibles a sesgos y vulnerabilidades que comprometen su integridad y confiabilidad.

- **Impacto de las Debilidades:** Estas debilidades no solo pueden ser consecuencia de clasificaciones erróneas o decisiones subóptimas, sino que también pueden ser **explotadas de forma maliciosa** para manipular los resultados del algoritmo, poniendo en riesgo la seguridad y la privacidad de los usuarios.
- **Importancia de la Auditoría:** Reconocer y abordar estas vulnerabilidades es esencial para crear sistemas de *Machine Learning* seguros y justos. Esto implica una auditoría minuciosa de los algoritmos para identificar y mitigar

tanto **sesgos no intencionados** como **fallos de seguridad**, lo que mejora la robustez y fortalece la confianza del usuario.

2. Clasificación de los Ataques Adversarios

Un **ataque adversario** en el *Machine Learning* representa la **manipulación intencionada de la entrada** de un modelo con el fin de engañarlo y provocar una clasificación o predicción incorrecta. Estos ataques explotan fallos de seguridad específicos de los algoritmos que están diseñados para funcionar bajo condiciones operativas estándar.

A. Clasificación por Nivel de Conocimiento del Atacante

Los ataques adversarios se clasifican en función del conocimiento que el atacante tiene del modelo objetivo:

- **Ataque de Caja Blanca (*White Box Attack*):** Supone un **conocimiento completo** del modelo, incluyendo su arquitectura, parámetros (pesos) y gradientes.
- **Ataque de Caja Negra (*Black Box Attack*):** Opera con **conocimiento limitado**, generalmente solo los *inputs* y los *outputs* del modelo (a través de la API).
- **Ataque de Caja Gris (*Grey Box Attack*):** Se encuentra en un punto intermedio entre los dos tipos anteriores.

B. Clasificación por Objetivos del Ataque

Estos ataques también se categorizan según el momento y el objetivo que persiguen:

- **Ataques de Envenenamiento (*Poisoning*):** Buscan comprometer el modelo durante su fase de **entrenamiento**, corrompiendo el proceso de aprendizaje.
- **Ataques de Evasión:** Intentan eludir la detección en la **fase operativa** (una vez desplegado el modelo).
- **Ataques Exploratorios:** Buscan **extraer información** sobre el modelo, realizando una especie de ingeniería inversa para comprometer la confidencialidad.

Los ataques de envenenamiento pueden ser particularmente destructivos, mientras que los de evasión y los exploratorios amenazan la integridad y la confidencialidad de los sistemas de *Machine Learning* ya desplegados.

3. El Ataque FGSM: Manipulación Sutil

Un ataque adversario muy conocido es el método de **Signo del Gradiente Rápido** (*Fast Gradient Sign Method* o **FGSM**), el cual es un tipo de ataque de caja blanca.

A. Funcionamiento del FGSM

El FGSM es una técnica que busca engañar a los modelos de *Machine Learning* mediante la **alteración mínima** de sus entradas.

- **Uso del Gradiente:** El método utiliza el gradiente de error del modelo para determinar cómo modificar la entrada de forma que **maximice el error de predicción**.
- **Efecto:** El método es efectivo incluso con pequeñas perturbaciones en los datos de entrada, lo que puede llevar a **clasificaciones incorrectas** sin que los cambios sean perceptibles para el ojo humano, lo cual está muy relacionado con la manipulación de imágenes.

B. Ejemplo Ilustrativo del FGSM (Imágenes)

La imagen clásica del ataque FGSM ilustra el proceso base:

1. **Imagen Original (x):** El modelo clasifica correctamente una imagen (ejemplo: un panda) con una confianza moderada.
2. **Perturbación ($\epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$):** El atacante añade una **pequeña perturbación** (ϵ) calculada a partir del **signo del gradiente** ($\text{sign}(\nabla_x J)$). El gradiente indica la dirección en la que el error del modelo es máximo. Al tomar el signo, se indica la dirección en la que deben moverse los píxeles para aumentar la pérdida.
3. **Ejemplo Adversario (x'):** El resultado de sumar la perturbación a la imagen original.

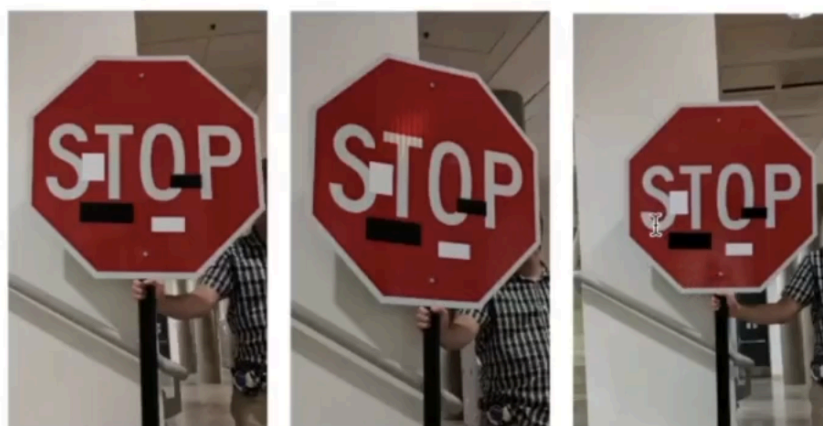
$$\begin{array}{ccc}
 \begin{array}{c} \text{Image of a panda} \\ x \\ \text{"panda"} \\ 57.7\% \text{ confidence} \end{array} & + .007 \times \begin{array}{c} \text{Image of noise} \\ \text{sign}(\nabla_x J(\theta, x, y)) \\ \text{"nematode"} \\ 8.2\% \text{ confidence} \end{array} & = \begin{array}{c} \text{Image of a gibbon} \\ x + \epsilon \text{sign}(\nabla_x J(\theta, x, y)) \\ \text{"gibbon"} \\ 99.3\% \text{ confidence} \end{array}
 \end{array}$$

El resultado final es una imagen que, aunque sigue pareciendo un panda para un humano, el modelo etiqueta incorrectamente (ejemplo: como un "gibbon") con una **confianza extremadamente alta** (ejemplo: 99.3% de confianza). La red neuronal ha sido completamente engañada por un ruido imperceptible.

C. Ataques en el Mundo Real

Existen casos muy conocidos de ataques adversarios aplicados al mundo físico:

- **Manipulación de Señales de Tráfico:** Se ha documentado el uso de **pegatinas tipo píxeles** colocadas en señales de tráfico (como las de **STOP**). Estas pegatinas, invisibles para el ojo humano o sutilmente colocadas, hacen que el modelo de visión por ordenador de un coche autónomo las interprete como otra señal (ejemplo: un límite de velocidad de 40), impidiendo que el coche realice la acción de detenerse.



4. Técnicas de Ataque más Complejas

Los ataques adversarios no se limitan a la modificación de píxeles, ni a las imágenes. También se pueden realizar contra **texto** y **audio** o contra cualquier tipo de dato de entrada.

- **Ataques de Escalado de Imágenes (*Scaling Attacks*):** Buscan explotar el proceso de **preprocesamiento** de redimensionamiento de imágenes. Un atacante manipula una imagen de entrada de manera que, una vez se redimensione, la salida sea una imagen completamente diferente. Por ejemplo, si la imagen de entrada es un pato, se podría insertar sutilmente la imagen de un gato de forma que, al escalar la imagen a otro tamaño, lo que el modelo vea sea el gato. La mitigación de estos ataques implica el uso de técnicas de preprocesamiento más robustas y menos predecibles, además de la validación de la entrada.
- **Ataques de Inversión de Pesos (Caja Negra):** Estos son ataques más complejos de **caja negra** que implican la aproximación o **clonación de los hiperparámetros internos** de un modelo de *Machine Learning* basándose exclusivamente en la entrada y la salida. Técnicas como el **aprendizaje por transferencia** y el *fine-tuning* de modelos pueden permitir a los atacantes replicar la funcionalidad del modelo objetivo sin acceso directo a su arquitectura o pesos.

5. Mitigación y Conclusiones

La protección contra los ataques adversarios en el aprendizaje automático requiere un enfoque **multifacético** que aborde tanto la seguridad del modelo como la integridad de los datos.

- **Refuerzo del Modelo:** La implementación de técnicas de **formación robusta** como la regularización y el **entrenamiento adversario** (reentrenamiento del modelo incluyendo ejemplos de entradas alteradas) mejora la resistencia del modelo frente a manipulaciones malintencionadas.

- **Sanitización de Datos:** Sanitizar y validar los datos de entrada son pasos vitales para asegurar que los datos procesados por el modelo no hayan sido alterados de manera adversaria.
- **Protección contra Inversión de Pesos:** Para protegerse contra la inversión de pesos del modelo, es crucial **limitar la información disponible** para los atacantes, lo que puede incluir la reducción de precisión en los *outputs* del modelo o la implementación de **mecanismos de tarificación** que limiten el número de consultas que un usuario pueda realizar.

El campo de la seguridad en el *Machine Learning* está en constante evolución. El desarrollo de métodos más avanzados para el análisis y la mitigación de ataques adversarios, como el FGSM, representa un área de investigación muy activa. La **anticipación y adaptación a las nuevas amenazas** serán vitales para proteger los sistemas de *Machine Learning* en el futuro.

El proyecto OMLASP subraya la **importancia crítica** de la seguridad y la imparcialidad de los algoritmos de *Machine Learning* y propone un marco para su auditoría y mejora continua. La colaboración e intercambio de conocimiento entre desarrolladores, especialistas de seguridad y expertos en *Machine Learning* son clave para avanzar hacia sistemas más seguros y confiables.

Introducción a la Sesión: FGSM en la Práctica

En esta sesión nos centraremos en el tema de los ataques **FGSM** (*Fast Gradient Sign Method*), pero abordados desde un punto de vista puramente **práctico**.

El objetivo central será crear un pequeño programa en Python que sea capaz de tomar una imagen y aplicarle de forma programática un ataque FGSM. Esto nos permitirá comprender la mecánica de cómo se generan estos ejemplos adversarios que comprometen la fiabilidad de los modelos de visión por ordenador.

Bloque de Código 1: Inicialización del Entorno y Carga del Modelo

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

model = tf.keras.applications.MobileNetV2(weights='imagenet',
include_top=True)
```

Análisis Detallado del Código

El objetivo de este primer bloque es preparar el entorno e inicializar la red neuronal que será el objetivo del ataque **FGSM** (*Fast Gradient Sign Method*).

1. Importación de Librerías:

- **import tensorflow as tf**: Importa la librería fundamental para *Deep Learning*. Es el marco que proporciona las herramientas para construir, cargar y manipular modelos de redes neuronales.

- `import numpy as np`: Importa la librería estándar para el manejo eficiente de arrays y operaciones numéricas.
- `import matplotlib.pyplot as plt`: Importa la librería para crear gráficos y, en este contexto, se utilizará para mostrar las imágenes antes y después de aplicar el ataque.

2. Carga del Modelo Pre-entrenado:

- `model = tf.keras.applications.MobileNetV2(...)`: Esta línea descarga e inicializa la arquitectura de red neuronal convolucional llamada **MobileNetV2**.
- `weights='imagenet'`: Indica que el modelo debe cargarse con pesos que ya han sido entrenados con el *dataset* de **ImageNet**. ImageNet es un conjunto de datos masivo con más de un millón de imágenes y mil categorías diferentes, lo que le permite al modelo reconocer una amplia variedad de objetos.
- `include_top=True`: Este parámetro asegura que se cargue la **red completa**, incluyendo las últimas capas de clasificación (la "parte superior" o *top*). Esto es crucial, ya que estas son las capas que finalmente realizan la clasificación de objetos basándose en las características extraídas.

3. Contexto del Modelo (MobileNetV2):

- **Propósito**: MobileNetV2 es una arquitectura diseñada específicamente para ser **ligera y eficiente**. Es ideal para aplicaciones en dispositivos móviles o entornos con recursos limitados.
- **Arquitectura**: Utiliza las llamadas capas de **cuello de botella** (*bottleneck*) para optimizar la eficiencia computacional y de memoria sin comprometer significativamente su rendimiento.

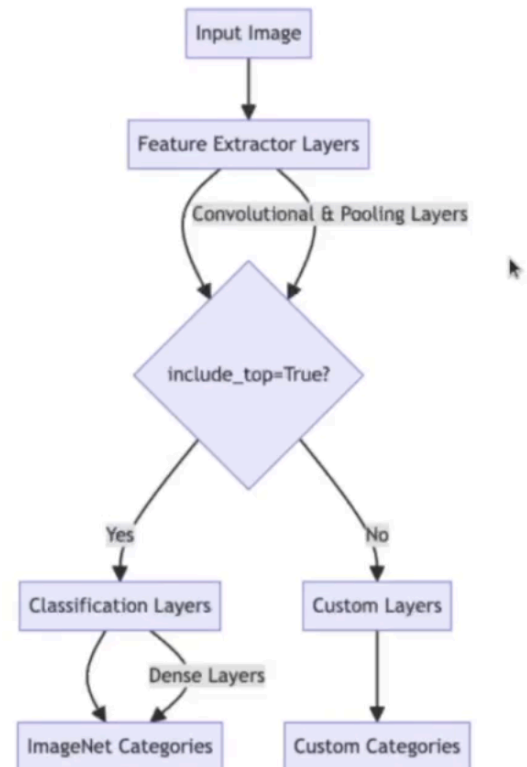
Bloque de Código 2: Carga y Visualización de la Arquitectura CNN

Aquí tienes el código del segundo bloque, incluyendo la línea que se añade a la importación y la celda de visualización:

```
import matplotlib.image as mpimg

img = mpimg.imread('CNN.jpg')
plt.figure(figsize=(10, 8))

plt.imshow(img)
plt.axis('off')
plt.show()
```



Análisis Detallado del Código y la Arquitectura CNN

I. Análisis Sintáctico del Código

1. **import matplotlib.image as mpimg**: Esta es la nueva línea de importación. Se utiliza para la **lectura de archivos de imagen** (en este caso, el diagrama de la arquitectura) antes de que Matplotlib los pueda dibujar.
2. **img = mpimg.imread('CNN.jpg')**: Lee el archivo de imagen grabado que contiene el diagrama de la arquitectura de la CNN y lo almacena en la variable **img**.
3. **plt.figure(figsize=(10, 8))**: Inicializa un lienzo gráfico con un tamaño específico para la visualización.
4. **plt.imshow(img)**: Muestra la imagen (el diagrama de la arquitectura) en el lienzo.

5. `plt.axis('off')`: Desactiva los ejes y las marcas numéricas para asegurar una visualización limpia del diagrama.
6. `plt.show()`: Renderiza el gráfico en la salida del *notebook*.

II. La Arquitectura de una Red Convolutiva (CNN)

La gráfica muestra la estructura de una CNN, que fundamentalmente tiene dos partes principales: el **extractor de características** y el **clasificador**.

1. Capas Extractoras de Características (*Feature Extractor Layers*)

- **Input Image**: El proceso comienza con la imagen de entrada.
- **Convolutional & Pooling Layers**: La imagen pasa por una serie de capas. Las **capas convolucionales** aplican filtros para detectar patrones (bordes, texturas), mientras que las **capas de pooling** reducen la dimensionalidad y retienen solo las características más importantes.

2. El Punto de Decisión: `include_top=True`?

Aquí entra en juego el parámetro que definimos al cargar MobileNetV2:

- **`include_top=True` (Sí)**: La red incluye las **Classification Layers** (capas de clasificación), que son las capas densas (*Dense Layers*) responsables de tomar las características extraídas y asignar la imagen a una de las categorías predefinidas en el entrenamiento original, como las **categorías de ImageNet**. **Este es el camino que hemos elegido** para nuestro ataque, ya que necesitamos que el modelo intente clasificar el objeto para poder medir su error.
- **`include_top=False` (No)**: La red omite las capas de clasificación preentrenadas. Esto permite añadir **Custom Layers** (capas personalizadas) diseñadas para tareas específicas con categorías diferentes a las de ImageNet.

III. Importancia del `include_top=True` para el FGSM

Al establecer `include_top=True`, estamos cargando la red completa que realiza la clasificación final en 1,000 categorías de ImageNet. Esto es esencial para el ataque FGSM, ya que el ataque necesita acceder a los **gradientes de la capa de clasificación** para determinar la dirección en la que el modelo comete el error.

Bloque de Código 2: Función para Cargar y Preprocesar la Imagen

```
def load_image(img_path):  
    img = tf.keras.preprocessing.image.load_img(img_path,  
target_size=(224, 224))  
    img = tf.keras.preprocessing.image.img_to_array(img)  
    img = np.expand_dims(img, axis=0)  
    img =  
tf.keras.applications.mobilenet_v2.preprocess_input(img)  
    return img
```

Análisis Detallado del Código

Esta función, `load_image`, encapsula todos los pasos de preparación necesarios para que una imagen sea compatible con el modelo **MobileNetV2** antes de la inferencia, asegurando que cumple con los requisitos de tamaño y formato del modelo.

1. `def load_image(img_path):`: Define la función y acepta la ruta de la imagen (`img_path`) como único argumento de entrada.

2. `img = tf.keras.preprocessing.image.load_img(img_path, target_size=(224, 224))`: Carga la imagen desde la ruta especificada. Crucialmente, esta línea **redimensiona** la imagen a 224×224 píxeles, ya que este es el tamaño de entrada específico que espera el modelo MobileNetV2. Es vital conocer las dimensiones de entrada requeridas por cualquier modelo de *Deep Learning*.
3. `img = tf.keras.preprocessing.image.img_to_array(img)`: Convierte la imagen cargada en un **array de NumPy**. Este formato es necesario para las operaciones matemáticas posteriores dentro de TensorFlow y NumPy.
4. `img = np.expand_dims(img, axis=0)`: Agrega una **dimensión extra** al inicio del *array*. El propósito es crear un *batch* (lote) de tamaño 1, transformando la imagen de un formato (altura, anchura, canales) a (1, altura, anchura, canales). Esta dimensionalidad es fundamental para poder procesar la imagen a través de la red neuronal.
5. `img = tf.keras.applications.mobilenet_v2.preprocess_input(img)`: Aplica las operaciones de **preprocesamiento específicas** del modelo MobileNetV2. Estas operaciones, que pueden incluir la normalización de los valores de los píxeles a un rango específico (como [-1,1] o [0,1]), son requisitos específicos del modelo que deben cumplirse para que la inferencia sea correcta. Si se utiliza otro modelo, este paso debe adaptarse.
6. `return img`: Devuelve el *array* de NumPy que representa la imagen, ahora completamente preprocesado y listo para ser utilizado como *input* para la red neuronal.

Bloque de Código 4: Carga de la Imagen Específica

```
img_path = 'ai_cat.jpg'
image = load_image(img_path)
```

Análisis Detallado del Código

Este bloque tiene un propósito muy directo: **identificar la imagen objetivo** y utilizar la función de preprocesamiento definida anteriormente.

1. **`img_path = 'ai_cat.jpg'`**: Define una variable `img_path` que contiene el nombre del archivo de imagen que queremos procesar. Esta imagen, que se supone que contiene la imagen de un gato, será el objetivo de nuestro ataque adversario.
2. **`image = load_image(img_path)`**: Llama a la función `load_image` que definimos en el bloque anterior. El propósito de esta llamada es cargar la imagen, redimensionarla a 224×224 píxeles, convertirla a un *array* de NumPy, añadir la dimensión del *batch* y aplicar el preprocesamiento específico de MobileNetV2. El resultado, ya listo para el modelo, se guarda en la variable `image`.

Bloque de Código 5: Definición de la Función de Ataque FGSM

```
def fgsm_attack(image, epsilon, data_grad):  
    sign_of_grad = tf.sign(data_grad)  
    perturbed_image = image + epsilon * sign_of_grad  
    perturbed_image = tf.clip_by_value(perturbed_image, -1, 1)  
    return perturbed_image
```

Análisis Detallado del Código

Esta función define la lógica del ataque **FGSM (Fast Gradient Sign Method)**, cuyo objetivo es generar una **imagen adversaria** que maximice el error de clasificación del modelo con una perturbación mínima.

1. **def fgsm_attack(image, epsilon, data_grad):**: Define la función que acepta tres argumentos clave:
 - **image**: La imagen original de entrada.
 - **epsilon**: Una pequeña constante que controla la **magnitud** o la fuerza de la perturbación que se va a añadir a la imagen. Este valor es crucial para el equilibrio entre el engaño y la invisibilidad.
 - **data_grad**: El gradiente del error del modelo con respecto a la imagen de entrada.
2. **sign_of_grad = tf.sign(data_grad)**: Calcula el **signo** del gradiente. El gradiente apunta en la dirección de la máxima pérdida o error. Al tomar solo el signo, esta línea determina la dirección en la que deben moverse los píxeles de la imagen para maximizar el error de clasificación del modelo.
3. **perturbed_image = image + epsilon * sign_of_grad**: Esta línea ejecuta el ataque central de FGSM. Suma la imagen original (**image**) con la perturbación calculada. La perturbación es el signo del gradiente, escalado por la magnitud **epsilon**. Esta operación asegura que el pequeño cambio

aplicado a cada píxel vaya en la dirección de aumentar el error de clasificación.

4. `perturbed_image = tf.clip_by_value(perturbed_image, -1, 1)`: Recorta los valores de los píxeles de la imagen modificada (`perturbed_image`) para que se mantengan dentro del rango válido de **-1 a 1**. Esto es necesario porque el modelo MobileNetV2 fue normalizado a este rango, y asegura que la imagen modificada permanezca normalizada antes de ser procesada por el modelo.
5. `return perturbed_image`: Devuelve la imagen adversaria, ahora modificada con el ruido diseñado para engañar al modelo.

Bloque de Código 6: Cálculo del Gradiente

```
label_index = 0
label = tf.one_hot(label_index, model.output_shape[-1])
label = tf.reshape(label, (1, model.output_shape[-1]))
image_tensor = tf.convert_to_tensor(image)
loss_object = tf.keras.losses.CategoricalCrossentropy()

with tf.GradientTape() as tape:
    tape.watch(image_tensor)
    prediction = model(image_tensor)
    loss = loss_object(label, prediction)
```

Análisis Detallado del Código

Este bloque prepara el **ataque adversario** al definir una etiqueta objetivo (que puede ser incorrecta) y calcular la dirección exacta (*el gradiente*) en la que la imagen debe ser modificada para maximizar el error del modelo.

1. Definición de la Etiqueta Objetivo

- `label_index = 0`: Define la posición de la **etiqueta objetivo**. En el contexto del ataque adversario, esta etiqueta seleccionada puede ser **deliberadamente incorrecta** para engañar al modelo.
- `label = tf.one_hot(label_index, model.output_shape[-1])`: Crea una representación *one-hot* (vector) de la etiqueta objetivo. Este vector tendrá un valor de **1** en la posición de la etiqueta objetivo (0) y ceros en el resto. La dimensión del vector está determinada por el número total de clases del modelo (`model.output_shape[-1]`), que en MobileNetV2 son 1000 categorías de ImageNet.
- `label = tf.reshape(label, (1, model.output_shape[-1]))`: Remodela el vector de etiqueta para asegurar que su forma sea compatible con el formato de *batch* esperado por el cálculo de la pérdida.

2. Preparación de Datos y Pérdida

- `image_tensor = tf.convert_to_tensor(image)`: Convierte la imagen de entrada, que está en formato NumPy, en un **tensor**. Esto es la estructura de datos que TensorFlow requiere para realizar operaciones a través de la red neuronal.
- `loss_object = tf.keras.losses.CategoricalCrossentropy()`: Define la función de pérdida a utilizar. Se instancia la **pérdida categórica cruzada**, la cual se usa en problemas de clasificación para medir el error de predicción. Compara la distribución de probabilidad predicha por el modelo con la distribución de probabilidad real (el vector *one-hot*).

3. Cálculo de la Dirección del Error (**GradientTape**)

- `with tf.GradientTape() as tape:`: Inicializa un contexto de `tf.GradientTape`. Esto funciona como una "cámara de seguridad" de TensorFlow, que **registra todas las operaciones matemáticas** que se ejecutan dentro del bloque.

- `tape.watch(image_tensor)`: Le indica a la cinta de gradiente que debe rastrear las operaciones con respecto al tensor de la imagen.
- `prediction = model(image_tensor)`: Realiza la clasificación de la imagen a través del modelo.
- `loss = loss_object(label, prediction)`: Calcula el valor numérico que representa el error (*pérdida*) del modelo al comparar su `prediction` con la `label` objetivo.
- `gradient = tape.gradient(loss, image_tensor)`: Esta línea se ejecuta **fuera** del bloque `with`. Utiliza el registro de operaciones (el `tape`) para calcular el **gradiente de la pérdida** con respecto al tensor de la imagen. El resultado, `gradient`, es un tensor que indica la dirección exacta en la que se debe modificar la imagen de entrada para **aumentar al máximo la pérdida** (es decir, confundir al modelo). Esta es la clave del ataque adversario.

Bloque de Código 7: Cálculo Final del Gradiente

```
gradient = tape.gradient(loss, image_tensor)
```

Análisis Detallado del Código

Esta línea, aunque breve, es el **paso final y crucial** dentro del proceso de cálculo de gradientes iniciado en el bloque anterior.

1. **Contexto:** Esta línea se ejecuta inmediatamente después de que se ha definido el error (`loss`) dentro del contexto de la **cinta de gradiente** (`tf.GradientTape`).
2. **Sintaxis:** Utiliza el método `.gradient()` de la cinta de gradiente.
3. **Argumentos:** Acepta dos argumentos clave:

- **loss**: La función de pérdida que se calculó en el bloque anterior. Este es el valor que queremos maximizar para engañar al modelo.
 - **image_tensor**: El tensor de la imagen de entrada.
4. **Propósito**: La función calcula el gradiente de la pérdida con respecto a la imagen original. Este gradiente es un tensor que indica exactamente **cómo deberían cambiar los valores de los píxeles** de la imagen para maximizar el error de clasificación del modelo. El resultado se almacena en la variable **gradient**, la cual será utilizada en la función de ataque FGSM para construir el ruido adversario.

Bloque de Código 8: Visualización de la Imagen Original

```
plt.imshow(tf.squeeze(image + 1) / 2)
plt.show()
```

Análisis Detallado del Código

Este bloque de código tiene el propósito de **mostrar la imagen original** antes de que se le aplique el ataque adversario, sirviendo como punto de referencia visual.

1. **plt.imshow(...)**: Esta función de Matplotlib es la encargada de dibujar la imagen en la salida.
2. **tf.squeeze(image + 1) / 2**: Esta expresión se encarga de preparar el *array* de la imagen para su visualización.
 - **image + 1**: La imagen original (**image**) fue normalizada previamente al rango de $[-1, 1]$ para ser compatible con MobileNetV2. Sumarle **1** desplaza el rango de valores a $[0, 2]$.

- `... / 2`: Dividir el resultado entre `2` reescala la imagen al rango de `[0,1]`. Este rango es el formato estándar que Matplotlib requiere para mostrar imágenes correctamente.
 - `tf.squeeze(...)`: Elimina las dimensiones de tamaño 1 del *array*, como la dimensión de *batch* que se añadió en el preprocesamiento, dejando el *array* en el formato tridimensional estándar para una imagen (altura, anchura, canales).
3. `plt.show()`: Renderiza y muestra la imagen procesada en el *notebook*.

Análisis del Output

El *output* de este bloque es la propia imagen:

- Se muestra la **imagen original del gato astronauta**, que fue generada con una herramienta de Inteligencia Artificial (DALL-E 3).
- Esta visualización **confirma que la imagen se cargó y preprocesó correctamente**, y es la base sobre la cual se aplicarán las perturbaciones adversarias en los siguientes pasos.

Bloque de Código 9: Aplicación del Ataque FGSM y Visualización

```
epsilon = 0.01
perturbed_image = fgsm_attack(image, epsilon, gradient)
```

(El bloque se repite con $\epsilon=0.40$ para la segunda demostración, y se muestra con el siguiente código.)

```
plt.imshow(tf.squeeze(perturbed_image+1)/2)
plt.show()
```

Análisis Detallado del Código y el Output

Este bloque aplica la lógica del ataque FGSM definida anteriormente y lo visualiza, demostrando el concepto clave de buscar un equilibrio entre el nivel de perturbación (el ruido) y la invisibilidad para el ojo humano.

1. **`epsilon = 0.01`**: Esta línea define el hiperparámetro **épsilon (ϵ)** con un valor inicial muy bajo. Este valor controla la **magnitud** o la intensidad del ruido adversario que se añade a la imagen.
2. **`perturbed_image = fgsm_attack(image, epsilon, gradient)`**: Llama a la función de ataque FGSM. Utiliza la imagen original, el valor ϵ (en este caso, 0.01) y el gradiente previamente calculado para generar la **imagen adversaria** modificada.
3. **Visualización (`plt.imshow...`)**: El código subsiguiente muestra la imagen alterada.

Análisis del Output (Variación del Épsilon)

La demostración se basa en la comparación visual al cambiar el valor de ϵ :

- **Caso 1: Épsilon bajo (0.01):**
 - **Output:** La imagen perturbada se ve **prácticamente idéntica a la original**.
 - **Conclusión:** La alteración es apenas perceptible, demostrando que el ruido adversario, aunque matemáticamente calculado, es **invisible para el ojo humano**. Este es el escenario ideal si es suficiente para engañar al modelo.
- **Caso 2: Épsilon alto (0.40):**
 - **Output:** La imagen se vuelve **visiblemente ruidosa** y distorsionada.
 - **Conclusión:** Se incrementa la perturbación para mostrar el efecto del ruido introducido por el ataque FGSM. Aunque esta modificación es **obvia**, no es el objetivo de un ataque adversario real.

Bloque de Código 10: Extracción y Visualización del Ruido Adversario

```
noise = perturbed_image - image
min_val = tf.reduce_min(noise)
max_val = tf.reduce_max(noise)
scaled_noise = (noise - min_val) / (max_val - min_val)

plt.subplot(1, 3, 3)
plt.title('Noise')
plt.imshow(tf.squeeze(scaled_noise))
plt.axis('off')
```

Análisis Detallado del Código

Este bloque tiene el propósito de **aislar y visualizar** la perturbación exacta que se añadió a la imagen original, lo que permite ver el ruido adversario que confunde a la Red Neuronal Convolutiva (CNN).

1. **noise = perturbed_image - image**: Calcula la diferencia entre la imagen modificada (**perturbed_image**) y la imagen original (**image**). El resultado es un tensor que contiene únicamente el **ruido adversario**.
2. **min_val = tf.reduce_min(noise)** y **max_val = tf.reduce_max(noise)**: Estas líneas encuentran los valores mínimo y máximo dentro del tensor de **noise**. Esto se hace porque el ruido es una diferencia entre dos imágenes normalizadas y sus valores pueden estar fuera del rango [0,1] necesario para la visualización.
3. **scaled_noise = (noise - min_val) / (max_val - min_val)**: Esta línea **reescala (normaliza)** el tensor del ruido. Ajusta los valores de **noise** al rango [0,1] utilizando los valores mínimos y máximos calculados. Este paso asegura que el ruido pueda ser visualizado correctamente por Matplotlib.

4. `plt.subplot(1, 3, 3)`: Crea un subgráfico en la posición 3 de una fila y 3 columnas, permitiendo que el ruido se muestre junto a la imagen original y la imagen adversaria.
5. `plt.title('Noise')` y `plt.axis('off')`: Asigna el título "Noise" y desactiva los ejes para una visualización limpia.
6. `plt.imshow(tf.squeeze(scaled_noise))`: Muestra el ruido adversario escalado.

Análisis del Output

El *output* del código es la visualización del tensor de ruido, etiquetado como "Noise".

- **Interpretación:** La imagen resultante es un patrón de **ruido multicolor**, aparentemente aleatorio, que no se asemeja a ningún objeto reconocible. Este es el mapa exacto de la alteración que el atacante ha añadido a la imagen original.

Este proceso de visualización del ruido es altamente educativo, ya que permite entender cuál es la **alteración mínima** y matemáticamente calculada que confunde a la CNN, demostrando la naturaleza del ataque FGSM. Puedes usar esta base de código para experimentar con diferentes valores de *epsilon* y modelos, intentando engañar a otros sistemas de reconocimiento en línea.

Conclusión: La Fragilidad y la Robustez en la Seguridad de la IA

La sesión práctica sobre el ataque FGSM (*Fast Gradient Sign Method*) ha sido un recordatorio potente de la fragilidad inherente en los modelos de aprendizaje, particularmente en los sistemas de visión por ordenador.

A través de pequeñas, pero muy puntuales, modificaciones en la entrada de datos que el ojo humano casi no puede percibir, es totalmente posible engañar a un

modelo para que realice clasificaciones incorrectas. Esto se demostró buscando el punto intermedio de la auditoría: encontrar el valor de ϵ (épsilon) más bajo posible que sea suficiente para confundir al modelo (por ejemplo, haciendo que clasifique un "panda" como "gibbon" o "nematode") sin que la alteración sea notoria para un observador humano.

Esta técnica no solo pone de manifiesto la importancia de la seguridad en la IA y los potenciales riesgos en aplicaciones críticas (como los vehículos autónomos o los sistemas de vigilancia), sino que también enfatiza la necesidad urgente de crear modelos que sean mucho más robustos y fiables.

Estrategias de Defensa contra Ataques Adversarios

(FGSM)

La fragilidad inherente a los modelos de visión por ordenador exige defensas robustas. Las siguientes son cinco técnicas clave para mitigar la efectividad de los ataques adversarios, como el FGSM:

1. **Entrenamiento Adversario (*Adversarial Training*):** Esta técnica consiste en mejorar la **robustez** del modelo incorporando intencionalmente imágenes perturbadas o modificadas dentro del conjunto de entrenamiento. Al exponer el modelo a ejemplos de ataque durante el entrenamiento, se aumenta su capacidad para clasificar correctamente incluso en presencia de alteraciones sutiles.
2. **Transformaciones de Entrada (*Input Transformations*):** Se trata de aplicar transformaciones a las imágenes **antes** de que sean procesadas por el modelo, con el fin de mitigar el efecto de las perturbaciones. Ejemplos de estas técnicas incluyen el **recorte de imágenes** (*image cropping*), la **adición de ruido** o la aplicación de filtros, que buscan alterar las modificaciones maliciosas sin cambiar significativamente el contenido original.
3. **Compresión de Características (*Feature Squeezing*):** Esta defensa reduce el espacio de búsqueda disponible para el atacante al **limitar la variabilidad de las entradas**. Por ejemplo, reducir la profundidad de color en las imágenes disminuye las variaciones que el ataque puede explorar, dificultando la generación de perturbaciones que sean a la vez efectivas y sutiles.
4. **Ensamblaje de Modelos (*Model Ensembling*):** Consiste en utilizar **múltiples modelos** en conjunto para hacer la predicción final. La idea es que, si bien un atacante podría engañar a un modelo individual, es menos probable que consiga engañar a varios modelos con arquitecturas o entrenamientos diferentes, aportando una capa adicional de seguridad.
5. **Sistemas de Detección de Anomalías:** Estos sistemas están diseñados para **identificar y rechazar entradas sospechosas** que puedan indicar un

intento de ataque. Esto se logra analizando las características de las imágenes para detectar patrones inusuales o alteraciones que difieran de forma significativa de los ejemplos típicos vistos durante el entrenamiento. Este enfoque se centra en la **vigilancia y el filtrado proactivo** de posibles amenazas antes de que comprometan la decisión del modelo, lo cual es esencial para una estrategia preventiva.

La implementación práctica a continuación se basará justamente en este último punto (Sistemas de Detección de Anomalías).

Bloque de Código 1: Implementación de la Detección de Ataques FGSM

Aquí tienes la definición de la función de detección de anomalías, que es el núcleo de esta sesión práctica:

```
def detect_fgsm_attack(original_image, perturbed_image,
threshold=0.5):
    # Predict the class of the original image.
    original_prediction = model.predict(original_image)

    # Predict the class of the perturbed image.
    perturbed_prediction = model.predict(perturbed_image)

    # Calculate the difference between the predictions.
    prediction_difference = np.abs(original_prediction -
perturbed_prediction)

    # Check for significant discrepancy.
    discrepancy = np.max(prediction_difference) > threshold

    return discrepancy
```

Análisis Detallado del Código (`detect_fgsm_attack`)

Esta función implementa un sistema de **Detección de Anomalías** basado en la **discrepancia de predicciones**, que es el enfoque elegido para la defensa.

1. **def detect_fgsm_attack(original_image, perturbed_image, threshold=0.5)::**
 - Define la función que acepta la **original_image** (la referencia) y la **perturbed_image** (la imagen a auditar).
 - El parámetro **threshold** (umbral) es crucial, estableciendo el nivel de diferencia de predicción que se considerará como un ataque. Este valor debe ajustarse para equilibrar los falsos positivos y falsos negativos.
2. **original_prediction = model.predict(original_image) y perturbed_prediction = model.predict(perturbed_image):**
 - **Propósito Clave:** Esta es la parte central. El modelo hace una predicción sobre qué objeto está viendo. Si bien la imagen modificada y la original son visualmente similares, se espera que, si hay manipulación, el modelo se confunda y clasifique la imagen modificada en una categoría diferente, o al menos cambie su nivel de confianza.
No se comparan píxeles ni histogramas, sino las categorías que el modelo "ve".
3. **prediction_difference = np.abs(original_prediction - perturbed_prediction):**
 - **Cálculo de la Diferencia:** Calcula la diferencia absoluta entre las probabilidades de predicción de la imagen original y la modificada. Una gran diferencia sugiere que el ruido adversario tuvo un impacto significativo en la clasificación.
4. **discrepancy = np.max(prediction_difference) > threshold:**
 - **Verificación:** Comprueba si la diferencia máxima entre las probabilidades (**np.max**) supera el **threshold** definido. Si lo supera, se concluye que hay una discrepancia significativa, implicando que la imagen modificada ha engañado al modelo.

5. **return discrepancy**: Devuelve un valor booleano (**True** si se detecta la anomalía, **False** en caso contrario).
-

Bloque de Código 2: Escenarios de Ataque y Análisis de Resultados

El código de demostración aplica la función `detect_fgsm_attack` después de modificar la imagen con dos niveles diferentes de ϵ (0.40 y 0.20).

Escenario A: Épsilon Alto ($\epsilon=0.40$)

- **Código de Ataque:** `epsilon = 0.40` y `perturbed_image = fgsm_attack(image, epsilon, gradient)`.
- **Imagen:** La imagen resultante es **visualmente evidente** el ataque, con ruido significativo.

Output de la Terminal:

FGSM attack detected.

- **Análisis:** Con un ϵ de 0.40, la perturbación es lo suficientemente agresiva como para causar una gran diferencia en las predicciones del modelo (pasa de ver "gato astronauta" a otra cosa). La función de detección de anomalías identifica esta discrepancia, concluyendo correctamente que **"FGSM attack detected."**

Escenario B: Épsilon Bajo ($\epsilon=0.20$)

- **Código de Ataque:** `epsilon = 0.20` y `perturbed_image = fgsm_attack(image, epsilon, gradient)`.
- **Imagen:** El ruido en la imagen es **menos útil** y más sutil.

Output de la Terminal:

No FGSM attack detected.

- **Análisis:** Al reducir ϵ a 0.20, el modelo ya **no detecta el ataque**. Esto tiene dos posibles interpretaciones en la auditoría:
 1. El ruido de $\epsilon=0.20$ **no fue lo suficientemente fuerte** para cambiar drásticamente la predicción del modelo (sigue viendo "gato astronauta"), por lo que la diferencia no superó el umbral.
 2. El umbral es demasiado alto, creando un **Falso Negativo** (un ataque existe, pero no se detecta).

El ejercicio demuestra que encontrar el **threshold** correcto es un **aspecto crítico** de la defensa, ya que un umbral bajo genera falsos positivos y un umbral alto genera falsos negativos.

Bloque de Código Final: Visualización Comparativa del Ataque FGSM

```
# Visualize the original, perturbed images and the difference.
plt.figure(figsize=(20, 8))

# Display the original image.
plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow(tf.squeeze(image+1)/2)
plt.axis('off')

# Display the perturbed image.
plt.subplot(1, 3, 2)
plt.title('Perturbed Image')
plt.imshow(tf.squeeze(perturbed_image+1)/2)
plt.axis('off')

# Display the difference image.
plt.subplot(1, 3, 3)
plt.title('Difference')
difference = perturbed_image - image
```

```
plt.imshow(tf.squeeze(difference / 2 + 0.5).numpy()) # Scale
difference to [0, 1] for displaying
plt.axis('off')

plt.show()
```

Análisis Detallado del Código y el Output

El propósito de este bloque es consolidar la fase de ataque visualizando en una única figura los tres componentes clave de la manipulación adversaria: la imagen de partida, el resultado del ataque, y el ruido generado.

Análisis Sintáctico del Código

1. **`plt.figure(figsize=(20, 8))`**: Crea un lienzo grande para asegurar que las tres imágenes se muestren una al lado de la otra de manera clara y visible.
2. **`# Display the original image. y plt.subplot(1, 3, 1)`**: Inicia el primer subgráfico. El `subplot(1, 3, 1)` posiciona la imagen en la primera columna de una cuadrícula de 1 fila por 3 columnas. Se muestra la imagen original, la cual ha sido **desnormalizada** (`tf.squeeze(image+1)/2`) del rango $[-1,1]$ al rango $[0,1]$ para su correcta visualización.
3. **`# Display the perturbed image. y plt.subplot(1, 3, 2)`**: Inicia el segundo subgráfico. Muestra la **imagen modificada** con el ruido adversario.
4. **`# Display the difference image. y plt.subplot(1, 3, 3)`**: Inicia el tercer subgráfico, que tiene como objetivo aislar el ruido.
 - **`difference = perturbed_image - image`**: Calcula el tensor exacto de la perturbación restando la imagen original de la imagen modificada.

- `plt.imshow(tf.squeeze(difference / 2 + 0.5).numpy())`: Esta es la visualización del ruido. El ruido se escala al rango $[0,1]$ con la operación $/ 2 + 0.5$ para que Matplotlib pueda renderizarlo correctamente.

5. `plt.axis('off')` y `plt.show()`: Desactiva los ejes en todos los subgráficos y renderiza la figura completa.

Análisis del Output (Gráfico)

El gráfico muestra las tres imágenes de izquierda a derecha, como se pretendía:



1. **Original Image:** La imagen del gato astronauta en su estado limpio.
2. **Perturbed Image:** La misma imagen con el ruido FGSM aplicado (visible como una fuerte distorsión de color, dado el valor de ϵ utilizado). Esta es la imagen que confunde al modelo.
3. **Difference:** Esta imagen visualiza el **ruido exacto** insertado en la imagen. Es un patrón de *pixels* aparentemente aleatorio, multicolor y denso, que es el **gradiente estructurado** diseñado para maximizar el error de la Red Neuronal Convolutiva (CNN).

Esta visualización finaliza la práctica, demostrando el mecanismo del ataque FGSM y la naturaleza de las perturbaciones adversarias.

Bloque de Código Final: Prueba de Detección (Control Positivo)

```
img_path = 'ai_cat.jpg'
image = load_image(img_path)

img_path = 'ai_cat.jpg'
perturbed_image = load_image(img_path)

attack_detected = detect_fgsm_attack(image, perturbed_image)

if attack_detected:
    print("FGSM attack detected.")
else:
    print("No FGSM attack detected.")
```

Análisis Detallado del Código

Este bloque tiene como objetivo realizar una **prueba de control**, verificando que el algoritmo de detección solo reporte un ataque cuando las imágenes son genuinamente diferentes.

1. Carga Doble de la Imagen Original:

- `img_path = 'ai_cat.jpg'` e `image = load_image(img_path)`: Carga la imagen original y la almacena en la variable `image`.
- `img_path = 'ai_cat.jpg'` y `perturbed_image = load_image(img_path)`: Carga la **misma imagen** y la almacena en la variable `perturbed_image`. Esto simula un escenario donde la imagen "modificada" es, de hecho, idéntica a la original.

2. Llamada a la Detección:

- `attack_detected = detect_fgsm_attack(image, perturbed_image)`: Llama a la función de detección de anomalías.

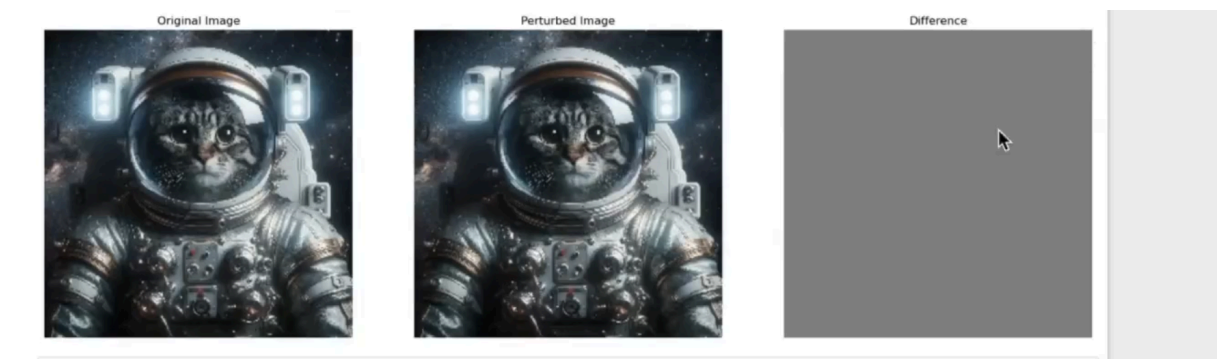
Como ambas variables contienen la misma imagen, la diferencia de predicción (`prediction_difference`) debería ser nula o insignificante, quedando por debajo del umbral (`threshold`).

3. Lógica Condicional:

- El bloque `if/else` imprime el resultado del booleano `attack_detected`.

Análisis del Output y la Visualización

El *output* y la visualización confirman el funcionamiento correcto del sistema de defensa:



Output de la Terminal:

No FGSM attack detected.

- El sistema reporta correctamente que **"No FGSM attack detected"**. Esto valida que el algoritmo de detección no produce un falso positivo cuando la imagen de entrada es idéntica a la imagen de referencia.
- **Visualización de las Tres Imágenes:**
 - La visualización comparativa (Original, Perturbed, Difference) muestra las dos primeras imágenes como idénticas.
 - La columna **Difference** (Diferencia) es completamente **gris**, lo que indica que no hay ninguna alteración ni ruido entre los valores de los píxeles, confirmando que la lógica de la detección es sólida para entradas idénticas.

Conclusión: Hacia una IA más Segura y Resiliente

Este ejercicio práctico sobre la detección de ataques FGSM (Fast Gradient Sign Method) resalta una faceta crucial en el ámbito de la IA: la **vulnerabilidad inherente** de los modelos de aprendizaje profundo ante manipulaciones apenas perceptibles.

La prueba de control que realizamos demostró que el algoritmo es **fiable**, ya que solo reacciona a cambios que superan un umbral específico. Este hecho subraya la importancia fundamental de aplicar esta detección **antes de la inferencia del modelo**, garantizando así que el sistema no se vea comprometido por ataques adversarios.

La capacidad de detectar y contrarrestar estos ataques es esencial. No solo plantea interrogantes sobre la seguridad y la integridad de las aplicaciones basadas en IA, sino que también subraya la necesidad de crear estrategias de defensa efectivas. Implementar modelos de detección de anomalías, como el que hemos visto, se convierte en un paso esencial hacia la creación de una IA más **robusta, confiable y segura**.